

EE-2003

Computer Organization and Assembly Language

Week 11 – Lecture 21

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

Outline

- 1 Quick Review
- 2 Local Variables in Stack
- 3 Bubble Sort: Global vs Local Variable

Quick Review: What was in the Last Week?

- Understanding Stack Pointer (SP)
- Nested Subroutine Calls in Bubble Sort- Example
- Understanding base Pointer & RET 4 :

Local Variables in Stack

- Local variables are temporary and exist only during subroutine execution.
- They are created when the subroutine starts and discarded when it ends.
- Unlike global variables, they do not occupy permanent memory space.
- The **stack** is the most convenient place to store local variables.

Creating Space for Local Variables:

- Space (gap) is created in the stack by adjusting the Stack Pointer (SP).
- Standard prologue for a subroutine with local variables:

```
push bp
mov bp, sp
sub sp, 2 ; creates 2-byte gap for a local word variable
```

- Local variables are accessed using **negative offsets** from BP.
- Parameters are accessed using **positive offsets** from BP.

Example: Bubble Sort – swapflag as a Local Variable

Example: Bubble Sort – swapflag as a Local Variable

- swapflag tracks if a swap occurred in an iteration.
- Instead of a global variable, it is now stored locally in stack space.
- Accessed at $[BP - 2]$ within the subroutine.

→ *BP acts as a fixed reference point: parameters at $BP+4$, $BP+6$; local variables at $BP-2$.*

Bubble Sort: Global vs Local Variable (Part 1)

Old Version: Used a global variable `swapflag` declared outside the subroutine.

- `swapflag` was defined in the data segment, taking permanent space.
- Any function could access or modify it, which is unsafe.

Example Code:

```
data: dw 60, 55
swapflag: db 0
...
mov byte [swapflag], 0
mov byte [swapflag], 1
cmp byte [swapflag], 1
```

Issue: Global variables break modularity and can interfere with other routines.

Bubble Sort: Local Variable on Stack (Part 2)

Updated Version: The swapflag is now made a **local variable** inside the subroutine.

Creating local variable:

```
push bp
mov bp, sp
sub sp, 2    ; create 2 bytes for local variable
```

- Creates a 2-byte gap on the stack below BP.
- Local variable accessed as [bp - 2].
- Memory is freed automatically when the function returns.

Accessing swapflag:

```
mov word [bp - 2], 0    ; reset flag
mov word [bp - 2], 1    ; set flag after swap
cmp word [bp - 2], 1    ; check flag
```

Stack Cleanup and Return (Part 3)

Code Example:

```
pop si
pop cx
pop bx
pop ax
mov sp, bp      ; restore SP
pop bp          ; restore old BP
ret 4            ; return and remove parameters
```

Explanation:

- The stack is restored before returning.
- `mov sp, bp` removes local variable space.
- `ret 4` pops the return address and 4 bytes of parameters.

Key takeaway:

- Local variables exist only during subroutine execution.
- They make functions modular and memory-efficient.

Main Differences: Global vs Local Variable (Part 4)

Global Variable	Local Variable (Stack-based)
Declared in data segment	Created dynamically on stack
Occupies permanent memory	Freed automatically after return
Accessible from anywhere	Accessible only inside subroutine
Less modular and unsafe	Safe, reusable, and function-specific
Accessed as [swapflag]	Accessed as [bp - 2]

Conclusion: Local variables improve modularity, save memory, and prevent unwanted interference between routines.

Bubble Sort with Local Variable (Part 1)

Program Setup and Swap Routine

```
[org 0x100]

jmp start

data:    dw    60, 55
; swapflag: db 0    ; Globals are bad! Let's make this local.

swap:
    push ax
    mov  ax, [bx + si]
    xchg ax, [bx + si + 2]
    mov  [bx + si], ax
    pop  ax
    ret
```

Note: This part defines the data and the swap routine — no global variable is used.

Bubble Sort with Local Variable (Part 2)

Subroutine Definition and Local Variable Creation

```
bubblesort:
    push bp
    mov  bp, sp
    sub  sp, 2          ; make space for local variable

    push ax
    push bx
    push cx
    push si

    mov  bx, [bp + 6]    ; address of data
    mov  cx, [bp + 4]    ; number of elements
    dec  cx
    shl  cx, 1
```

Explanation:

- The line 'sub sp, 2' reserves 2 bytes for the local variable.
- The local variable is accessed as '[bp - 2]'.

Bubble Sort with Local Variable (Part 3A)

Main Loop – Checking and Swapping Elements

```
mainloop:
    mov     si, 0
    mov     word [bp - 2], 0      ; local swap flag

innerloop:
    mov     ax, [bx + si]
    cmp     ax, [bx + si + 2]
    jbe     noswap
    call    swap
    mov     word [bp - 2], 1

noswap:
    add     si, 2
    cmp     si, cx
    jne     innerloop
```

- Local variable [bp - 2] tracks whether a swap occurred.
- After each comparison, elements are swapped if out of order.
- When no swaps occur, sorting is complete.

Bubble Sort with Local Variable (Part 3B)

Loop Completion and Stack Cleanup

```
    cmp    word [bp - 2], 1
    je     mainloop

    pop    si
    pop    cx
    pop    bx
    pop    ax
    mov    sp, bp
    pop    bp
    ret    4
```

Explanation:

- The loop continues if a swap occurred.
- `mov sp, bp` resets stack to before local variable space.
- `ret 4` returns and removes 4 bytes of parameters.

Key Point: Local variable `[bp - 2]` is automatically discarded once the subroutine ends.

Thank You!

Questions or Discussion?

- **Key Concepts Reviewed:**

- Subroutines use **CALL** and **RET** for modular code.
- **SP** tracks top of the stack; **BP** accesses parameters and locals.
- Local variables are stored on the stack and cleared automatically.
- Stack-based parameter passing enables nested and reusable functions.
- **RET** *n* returns control and cleans up the stack.

“Mastering stack handling is the foundation of structured assembly programming.”